[11]

Rebasing

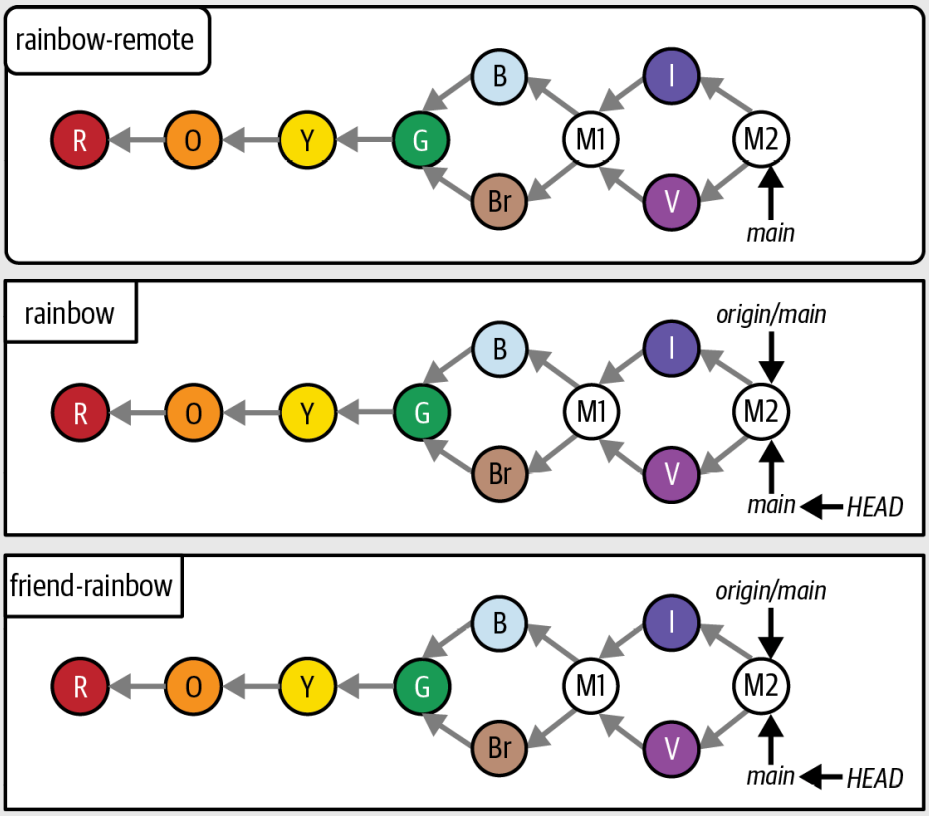
As you know, Git offers two main ways to integrate changes from one branch to another: merging and rebasing. In Chapters 5 and 9 you learned about fast-forward merges and three-way merges, respectively. In Chapter 10, you learned about merge conflicts and found out that they can arise both during a three-way merge and during a rebase.
In this chapter, you will learn about rebasing and walk through an example. I'll introduce the five stages of the rebase process and show you how to resolve merge conflicts during this process. We'll look at how merging and rebasing differ, and in which cases you may prefer to rebase instead of merge. I'll also introduce the golden rule of rebasing, which will help you determine when not to rebase. Additionally, you will learn a bit more about the staging area and how to use it as a rough draft space for organizing what you want to include in your next commit.

State of the Local and Remote Repositories

At the start of this chapter, you should have two local repositories called rainbow and friend-rainbow and one remote repository called rainbow-remote. All three of these repositories should contain the same commits and branches. As usual, when working through the examples in this chapter I recommend that you use two separate text editor windows and command line windows for the rainbow repository and the friend-rainbow repository.

Visualize it 11-1 shows the state of the local and remote repositories in the Rainbow project with all the commits that were made from Chapter 1to Chapter 10.

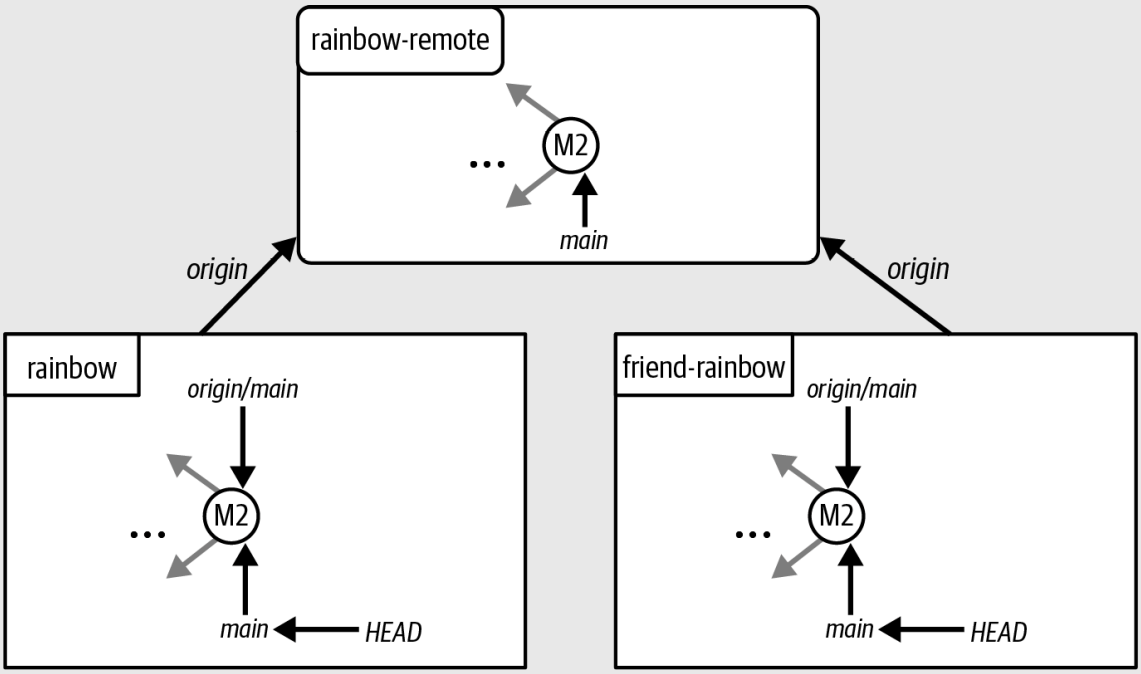
[VISUALIZE IT 11-1]



The Rainbow project at the start of Chapter 11 with all the commits since Chapter 1

To focus on the commits you are going to make in this chapter, from here on I will simplify the Visualize It diagrams and show only the last commit that was made on the main branch in all the repositories, which is the M2 merge commit. This representation is shown in Visualize it 11-2.

[VISUALIZE IT 11-2]



A simplified representation of the Rainbow project at the start of Chapter 11, showing just the last commit on the main branch in all the repositories

Integrating Changes in Git

Up until now, we have focused on merging as a way of integrating changes in Git. You have seen in practice that a fast-forward merge simply moves the branch pointer of the target branch to point to the latest commit, whereas a three-way merge creates a merge commit that ties the development histories of the source branch and the target branch together (and may sometimes also lead to merge conflicts).

If you look at the state of the various repositories in Visualize it 11-1, you can see that up until the green commit you had a linear project history, but after the green commit the commit history is nonlinear due to the merge commits created by the three-way merges. Some teams and individuals prefer to maintain a linear project history because they find it is more organized and simpler. You can use the process of rebasing to avoid three-way merges and merge commits and maintain a linear project history.

Rebasing takes all the work you have done in the commits on one branch and reapplies the work on another branch, creating entirely new commits. This can make it appear as

though you have created a branch from an entirely different commit than the original commit you created it from.

To carry out a rebase, you need to be on the branch you want to rebase. You use the git rebase command and pass in the name of the branch that you want to rebase onto.

[SAVE THE COMMAND]

git rebase *<branch_name>*

Reapply commits on top of another branch

Given that rebasing creates entirely new commits, this means it changes the commit history. It's important to be careful when using Git operations that do this. We'll discuss situations where it is not recommended to rebase a branch in "The Golden Rule of Rebasing" on page 223. First, however, I'm going to show you how rebasing may be used in a Git project, and you'll work through a hands-on example in the Rainbow project. Let's get started by exploring why you might want to take this approach.

Why Is Rebasing Helpful?

To illustrate how rebasing can help you avoid three-way merges and maintain a linear project history, let's revisit the example introduced in Example Book Project 9-1. Take a look at Example Book Project 11-1.

EXAMPLE BOOK PROJECT 11-1

Using the example scenario from Chapter 9, suppose my coauthor and I each make a different branch based on the main branch when it is pointing to commit B, to work on separate chapters. I make the chapter_five branch and my coauthor makes the chapter_six branch, as seen in Figure 11-1.

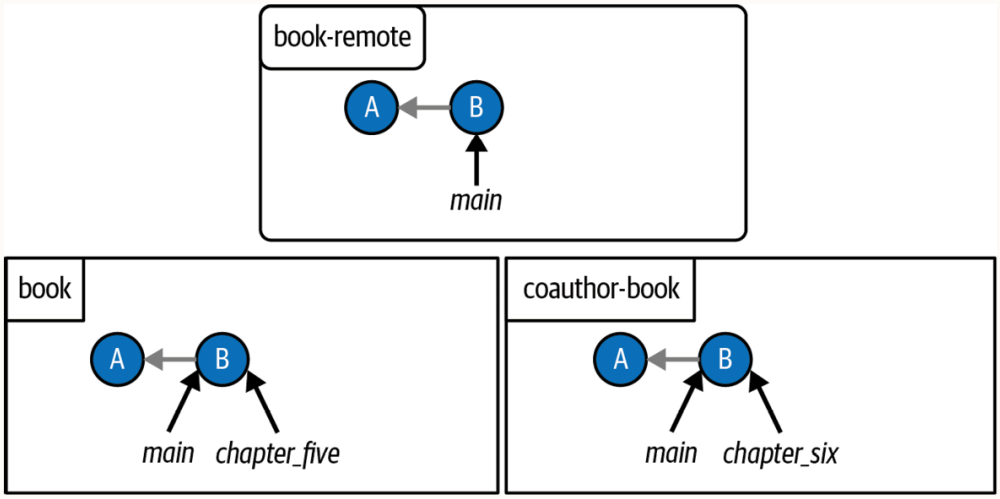


FIGURE 11-1

The Book project when my coauthor and I each make branches to work on different chapters

My coauthor works on chapter 6 of the book and makes commit C. They merge their work into the main branch of their local repository and push the updated main branch to the remote repository. At the same time, I work on chapter 5 and make commit D in my local repository. However, I have not yet merged or shared my work. These developments are illustrated in Figure 11-2.

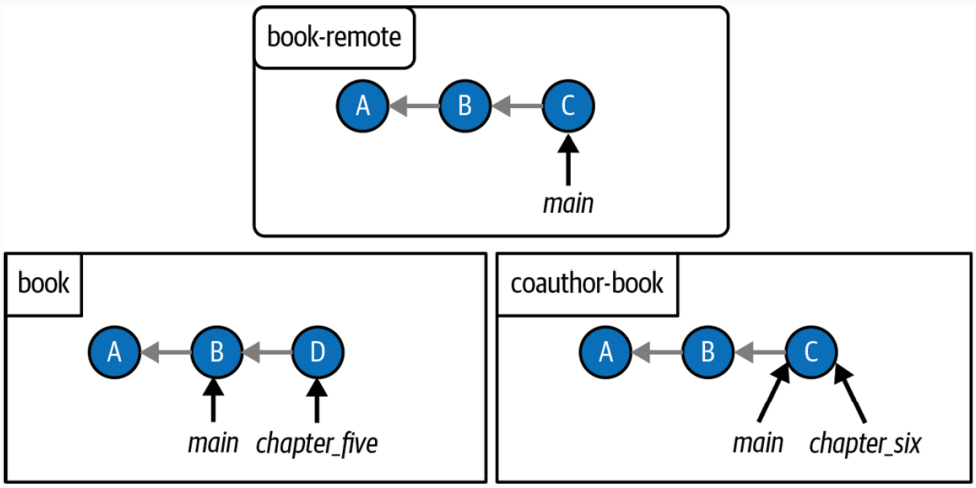


FIGURE 11-2

The Book project after my coauthor pushes their changes to the remote repository

When I decide that my changes to chapter 5 are ready to be merged into my local main branch, I now have a couple of options. One option is to fetch the changes on the remote main branch from the remote repository and then do a three-way merge to merge chapter_five into main, which will ultimately produce a merge commit (represented as commit M
in Figure 11-3). This is the option presented in Chapter 9.

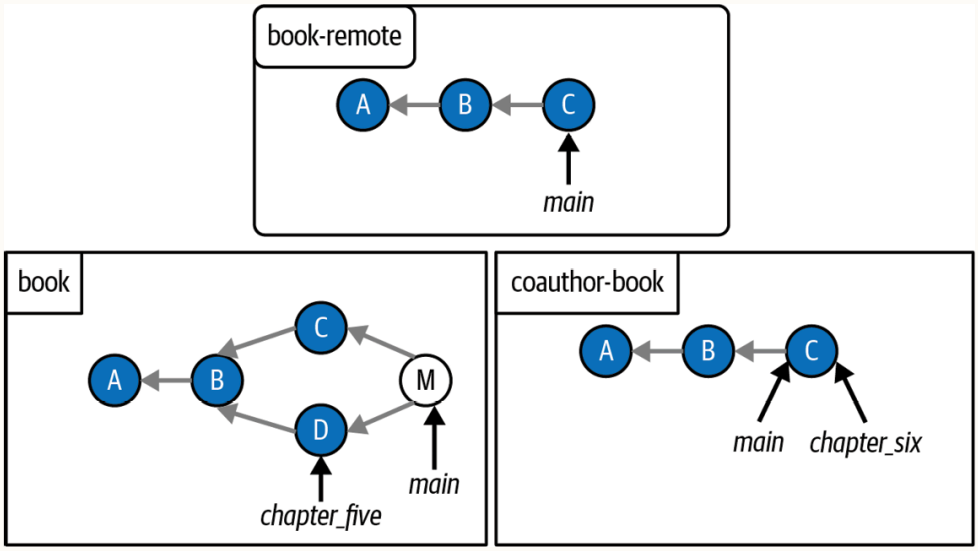


FIGURE 11-3

The Book project if I were to integrate the changes from the remote repository using a three-way merge

Another option is to pull the changes from the remote repository, which means my local main branch will update to point to commit C, with the intent of rebasing my branch afterward. Figure 11-4 shows the state of the Book project after I pull the changes from the remote main branch.

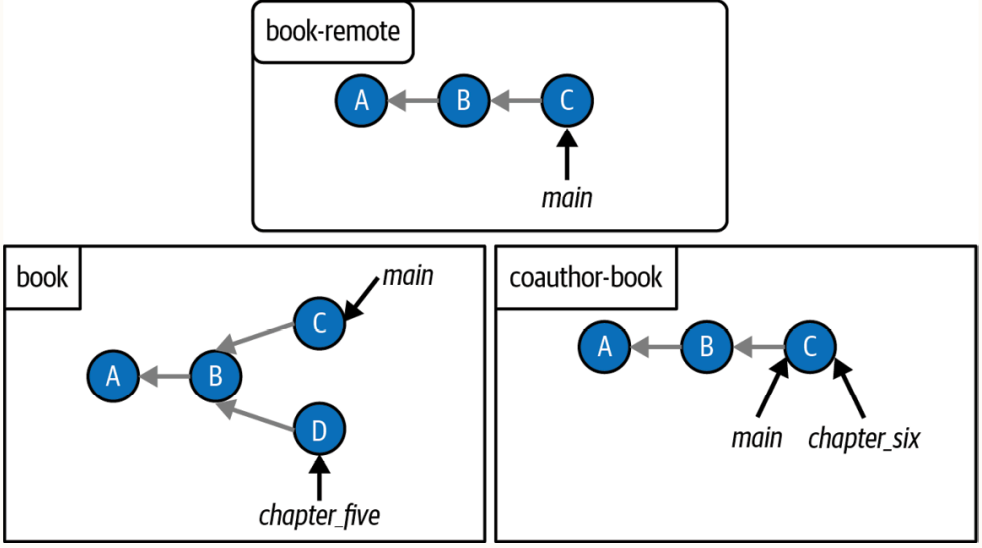


FIGURE 11-4

The Book project after I pull the changes from the remote main branch

Next, I can rebase my chapter_five branch on top of the updated main branch. As mentioned previously, rebasing will take all the commits I have made on the chapter_five branch and reapply them onto the main branch, creating entirely new commits. In this example, the only commit that I made on the chapter_five branch since it diverged from the main branch is commit D. When the branch is rebased, a new D′ commit will be created, as seen in Figure 11-5. The apostrophe (′) in the identifier indicates that this is a rebased commit.

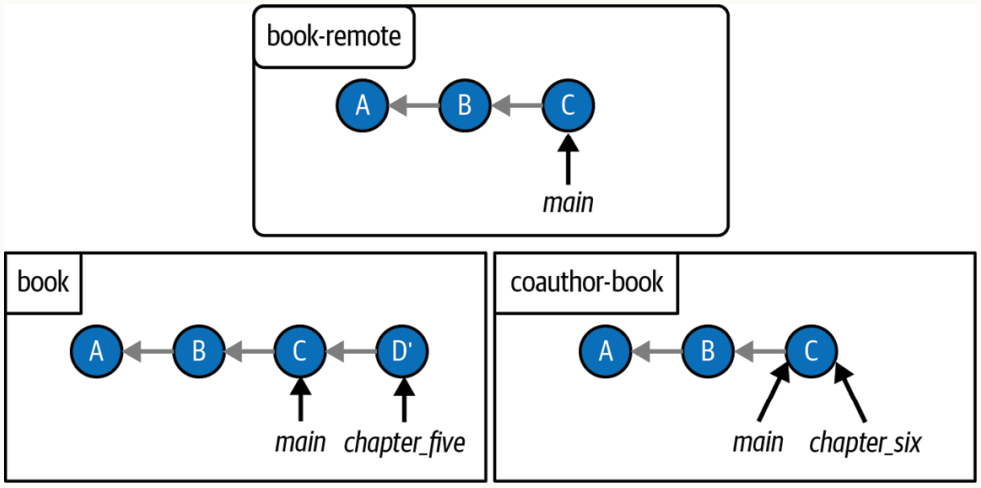


FIGURE 11-5

The Book project after I integrate the changes from the remote repository by rebasing my chapter_five branch onto the main branch

The D′ commit represents the new version of commit D that was created during the rebase process. It includes all the changes I made in the original D commit, but it is an entirely new commit with a new commit hash.

After I have rebased my chapter_five branch, I can merge it into main with a simple fast-forward merge, as seen in Figure 11-6.

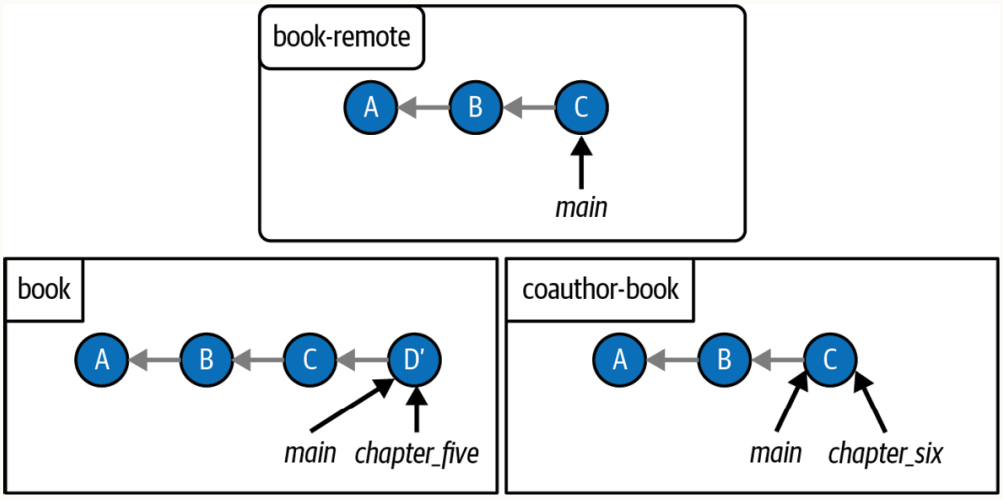


FIGURE 11-6

The Book project after I merge the chapter_five branch into the main branch with a simple fast-forward merge

By rebasing my branch I'm able to maintain a linear commit history and avoid making additional merge commits.

Note that the original D commit still exists in the commit history; however, it is no longer part of any of the branches in the book repository, which is why it is not shown in Figure 11-6. Rebasing a branch does not delete the commits on that branch; it simply recreates them. The old commits still exist in the commit history.

Now that you've seen a hypothetical example of how rebasing can be used to maintain a linear project history, let's create a situation in the Rainbow project where you have divergent development histories between two branches so you can go over a hands-on example.

Setting Up the Rebasing Example

To practice rebasing, you'll need to create divergent histories. You are going to make one commit in the rainbow repository and push it to the remote repository. Then your friend, without fetching the changes from the remote repository, is going to make two commits in their friend-rainbow repository. After they have made their commits, they will fetch the changes from the remote repository and decide to rebase their branch.

Go to Follow Along 11-1 to make a commit in the rainbow repository.

[FOLLOW ALONG 11-1]

1. Open the othercolors.txt in the rainbow project directory in a text editor window. Add “Gray is not a color in the rainbow.” on line 2, and save the file.

2. rainbow $ **git add othercolors.txt**

3. rainbow $ **git commit -m "gray"**
   [main 6f2cf36] gray
   1 file changed, 2 insertions(+), 1 deletion(-)

4. rainbow $ **git push**
   Enumerating objects: 5, done.
   Counting objects: 100% (5/5), done.
   Delta compression using up to 4 threads
   Compressing objects: 100% (3/3), done.
   Writing objects: 100% (3/3), 327 bytes | 327.00 KiB/s, done.
   Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
   To github.com:gitlearningjourney/rainbow-remote.git
   f10f972..6f2cf36 main -> main

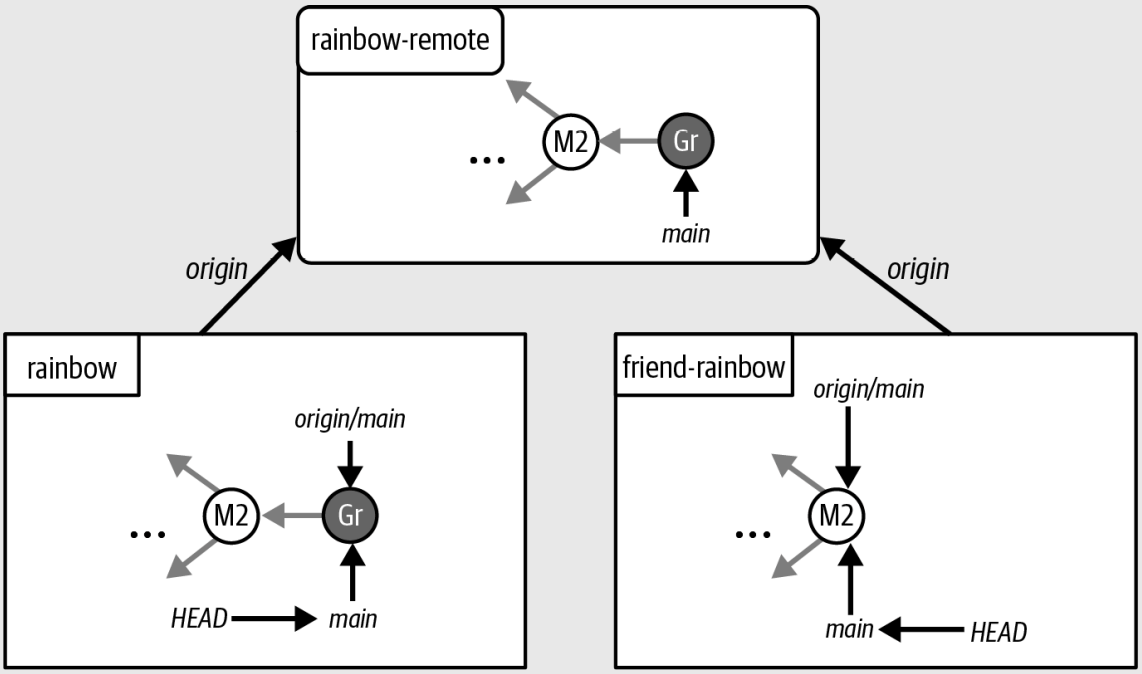
5. rainbow $ **git log**
   commit 6f2cf3698e6bf9078e8e0340ec9948f590405091 (**HEAD -> main**, **origin/main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sun Feb 20 09:27:58 2022 +0100
   gray
   commit f10f9725e3319af840a3d891ca8950436a219eb0
   Merge: 6ad5c15 9b0a614
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sun Feb 20 09:11:06 2022 +0100
   merge commit 2

What to notice:

- You have made the gray commit and pushed it to the remote repository.

This is illustrated in Visualize it 11-3.

[VISUALIZE IT 11-3]



The Rainbow project after you make the gray commit on the local main branch in the rainbow repository and push it to the remote repository

Next, your friend will add some work in the friend-rainbow repository on the same branch, which will lead to divergent histories between your main branch and your friend's main branch. This also provides a good opportunity to introduce some helpful features of the staging area.

Unstaging and Staging Files

In this section, your friend is going to make changes to both of the files in the friend-rainbow repository and add them both to the staging area. But then they will realize they want to make two separate commits for the different pieces of work. Therefore, they will have to *unstage* a file.

Let's take a look at Example Book Project 11-2, in which we revisit the scenario that we explored in Example Book Project 3-2 to see why we might need to unstage a file.

EXAMPLE BOOK PROJECT 11-2

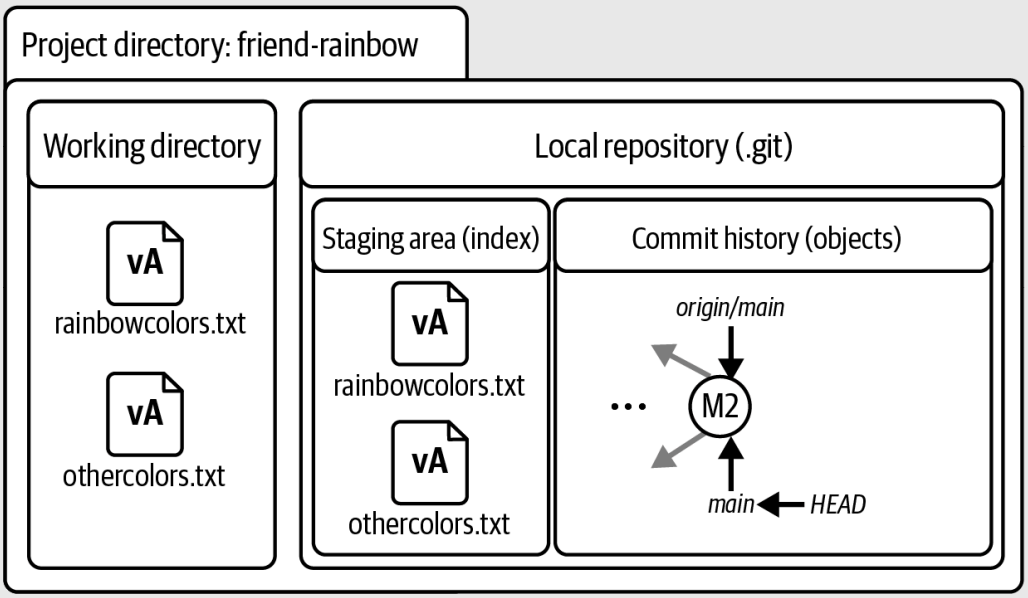
Suppose I work on chapters 1, 2, and 3 of my book, editing the files corresponding to those chapters: chapter_one.txt, chapter_two.txt, and chapter_three.txt. I add all the chapter files to the staging area, which means all the changes I have made to chapters 1, 2, and 3 will be saved in my next commit. My plan is for my editor to review all of my changes.

Before I make the commit, however, I realize that only the changes in chapter 2 are actually ready to be reviewed. In other words, I want only the changes that I made in chapter_two.txt to be included in my next commit. Since the staging area is a rough draft space where I can add and remove modified files in order to craft what will be included in the next commit, I'm free to remove the modified chapter_two.txt and chapter_three.txt files from this area, so that only the changes in the modified chapter_two.txt file are included in my next commit. This means my editor will review only those changes.

Now that you have an idea of why it is important to know how to unstage a file from the staging area, you're ready to practice unstaging a file in the Rainbow project.

To explore what happens in the upcoming example, we will focus on the friend-rainbow repository in the Visualize It diagrams, using the Git Diagram introduced in Chapter 2. The current state of the friend-rainbow repository is shown in Visualize it 11-4.

[VISUALIZE IT 11-4]



The friend-rainbow project directory before your friend edits any files

What to notice:

- The current versions of the rainbowcolors.txt file and othercolors.txt file that are in the working directory and staging area are represented as version A (vA).

Next, in Follow Along 11-2, your friend is going to make some changes to the files.

[FOLLOW ALONG 11-2]

1. In the friend-rainbow project directory in your text editor:
   In the othercolors.txt file, add “Black is not a color in the rainbow.” on line 2 and save the file.
   In the rainbowcolors.txt file, add “These are the colors of the rainbow.” on line 8 and save the file.

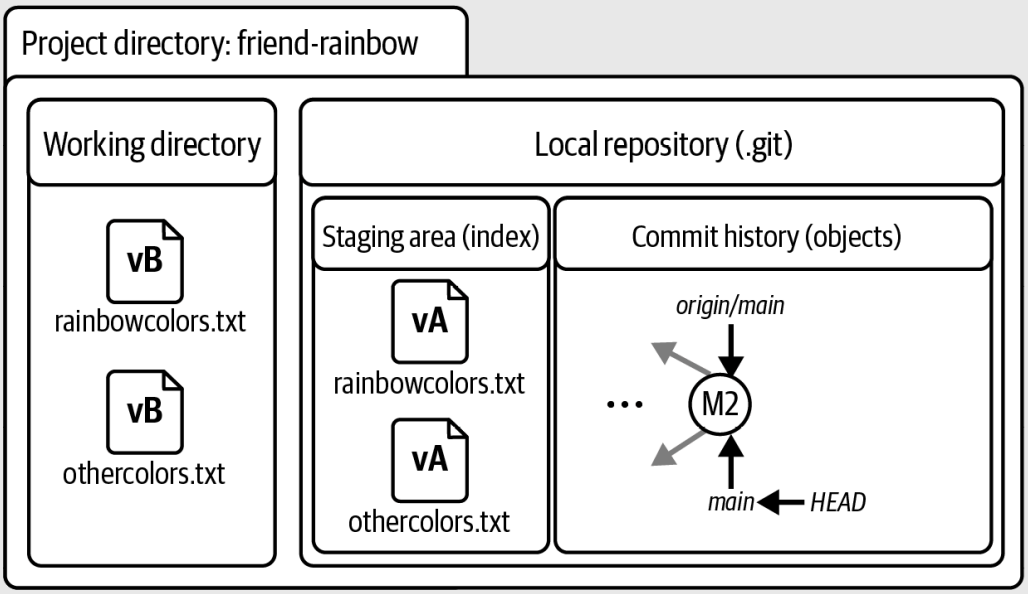
2. friend-rainbow $ **git status**
   On branch main
   Your branch is up to date with 'origin/main'.
   Changes not staged for commit:
   (use "git add <file>..." to update what will be committed)
   (use "git restore <file>..." to discard changes in working directory)
   modified: othercolors.txt
   modified: rainbowcolors.txt
   no changes added to commit (use "git add" and/or "git commit -a")

What to notice:

- Your friend edited both the rainbowcolors.txt file and the othercolors.txt file, and they are listed as modified files that have not been added to the staging area.

This is illustrated in Visualize it 11-5.

[VISUALIZE IT 11-5]



The friend-rainbow project directory after your friend edits the rainbowcolors.txt and othercolors.txt files in the working directory

What to notice:

- The modified versions of the rainbowcolors.txt file and the othercolors.txt file that your friend edited are represented as version B (vB).
- The versions of the othercolors.txt and rainbowcolors.txt files in the staging area (vA) are different from the versions in the working directory (vB).

Now, in Follow Along 11-3, your friend will add the updated files to the staging area to include them in the next commit.

[FOLLOW ALONG 11-3]

1. friend-rainbow $ **git add rainbowcolors.txt othercolors.txt**

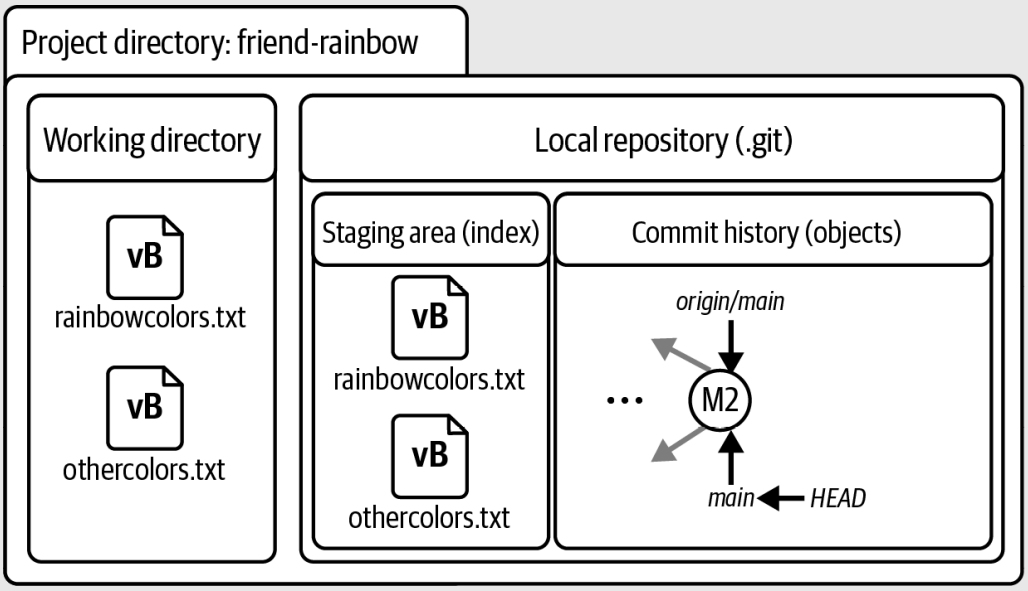
2. friend-rainbow $ **git status**
   On branch main
   Your branch is up to date with 'origin/main'.
   Changes to be committed:
   (use "git restore --staged <file>..." to unstage)
   modified: othercolors.txt
   modified: rainbowcolors.txt

What to notice:

- Your friend added both the modified rainbowcolors.txt file and the modified othercolors.txt file to the staging area.

This is illustrated in Visualize it 11-6.

[VISUALIZE IT 11-6]



The friend-rainbow project directory after your friend adds the updated versions of the rainbowcolors.txt and othercolors.txt files to the staging area

What to notice:

- The versions of the othercolors.txt and rainbowcolors.txt files in the staging area have changed from vA to vB.

Now, let's assume that your friend decides they want to make separate commits for the work they did adding the color black to the list of non-rainbow colors and the work they did commenting on the rainbow.

In “Introducing the Staging Area” on page 27, I mentioned that the staging area is like a rough draft space where you can add and remove modified files to craft what will be included in your next commit. Up until now, you have been using the git add command to add files to the staging area. Now you're going to see how to remove a modified file from the staging area, or in other words, change the version of the file in the staging area. In the git status command output in Follow Along 11-3, Git provides the instructions for how to unstage a file: (use "git restore --staged <file>..." to unstage). As this output indicates, to remove files from the staging area you can use the git restore command with the --staged option, passing in the names of any files you want to unstage, separated by spaces.

[SAVE THE COMMAND]

git restore --staged <filename>

Restore a file to another version of the file in the staging area

[NOTE]

If you have a version of Git that is older than version 2.23, then you won't have access to the git restore command. Your output may suggest you use the git reset command, which is another command that can unstage a file. In the upcoming Follow Along, in step 1, you will have to enter git reset HEAD rainbowcolors.txt instead of git restore --staged rainbowcolors.txt to unstage the rainbowcolors.txt file. Keep in mind that in Git there are often many different ways to achieve the same outcome.

In Follow Along 11-4, your friend will use the git restore command with the --staged option, as directed in the git status output in Follow Along 11-3, passing in the name of the rainbowcolors.txt file to unstage it. This means their next commit will include only the changes they made in the othercolors.txt file.

[FOLLOW ALONG 11-4]

1. friend-rainbow $ **git restore --staged rainbowcolors.txt**

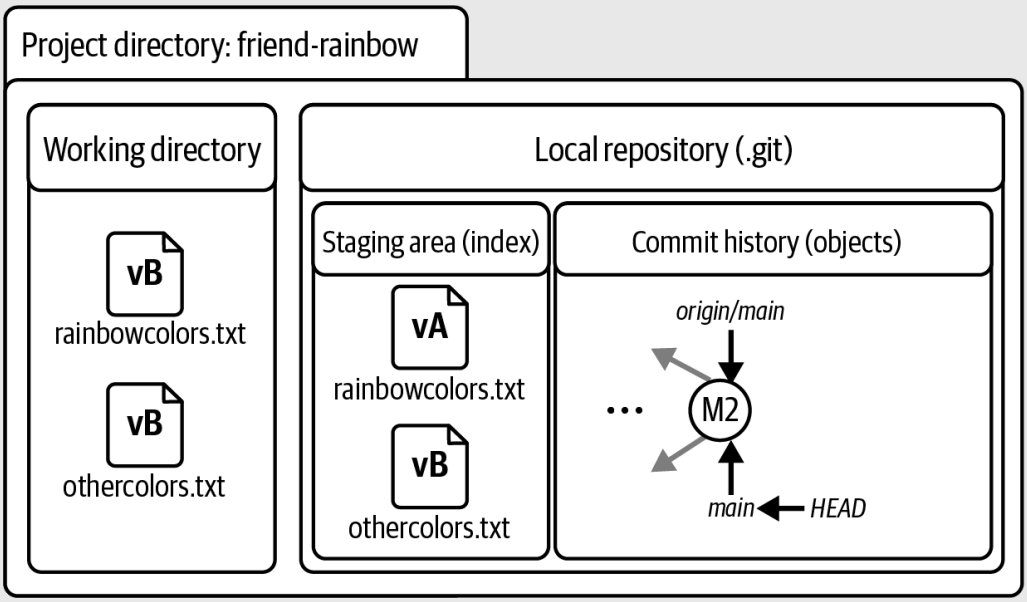
2. friend-rainbow $ **git status**
   On branch main
   Your branch is up to date with 'origin/main'.
   Changes to be committed:
   (use "git restore --staged <file>..." to unstage)
   modified: othercolors.txt
   Changes not staged for commit:
   (use "git add <file>..." to update what will be committed)
   (use "git restore <file>..." to discard changes in working directory)
   modified: rainbowcolors.txt

What to notice:

- The git status output shows that the updated version of the othercolors.txt file is still in the staging area, whereas the updated version of the rainbowcolors.txt file has been unstaged and is no longer in the staging area.

We illustrate this in Visualize it 11-7.

[VISUALIZE IT 11-7]



The friend-rainbow project directory after your friend unstages the rainbowcolors.txt file

What to notice:

- Your friend unstaged the rainbowcolors.txt file, so the version in the staging area went back to vA.
- The updated version of othercolors.txt (vB) is still in the staging area.
- The version of rainbowcolors.txt with your friend's most recent changes (vB) is still in the working directory.

Next, in Follow Along 11-5, your friend will make a commit that will include only the change to the othercolors.txt file.

[FOLLOW ALONG 11-5]

1. friend-rainbow $ **git commit -m "black"**
   [main 29bdadd] black
   1 file changed, 2 insertions(+), 1 deletion(-)

2. friend-rainbow $ **git status**
   On branch main
   Your branch is ahead of 'origin/main' by 1 commit.
   (use "git push" to publish your local commits)
   Changes not staged for commit:
   (use "git add <file>..." to update what will be committed)
   (use "git restore <file>..." to discard changes in working directory)
   modified: rainbowcolors.txt
   no changes added to commit (use "git add" and/or "git commit -a")

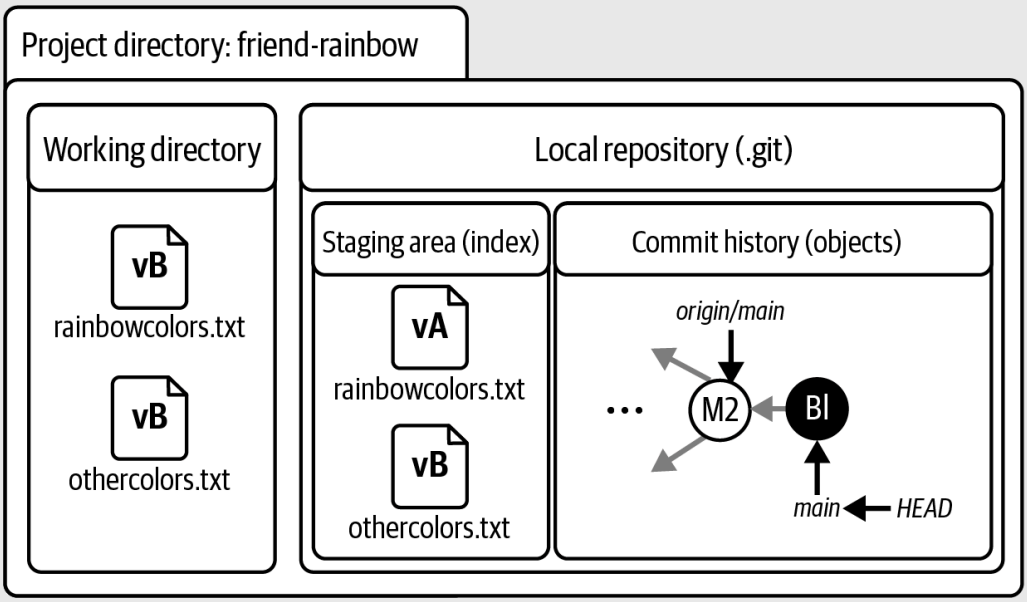
3. friend-rainbow $ **git log**
   commit 29bdadd50ddea41c75b476e776b6204a555b3d54 (**HEAD -> main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sun Feb 20 10:07:38 2022 +0100
   black
   commit f10f9725e3319af840a3d891ca8950436a219eb0 (**origin/main, origin/HEAD**)
   Merge: 6ad5c15 9b0a614
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sun Feb 20 09:11:06 2022 +0100
   merge commit 2

What to notice:

- Your friend made the black commit.
- In step 2, the git status output mentions that the rainbowcolors.txt file is a modified file in the working directory.

Visualize it 11-8 shows the state of the friend-rainbow project directory after Follow Along 11-5.

[VISUALIZE IT 11-8]



The friend-rainbow project directory after your friend makes the black commit

Now, in Follow Along 11-6, your friend will add the changes that they made to the rainbowcolors.txt file to the staging area and make another commit.

[FOLLOW ALONG 11-6]

1. friend-rainbow $ **git add rainbowcolors.txt**

2. friend-rainbow $ **git commit -m "rainbow"**
   [main 51dc6ec] rainbow
   1 file changed, 1 insertion(+), 1 deletion(-)

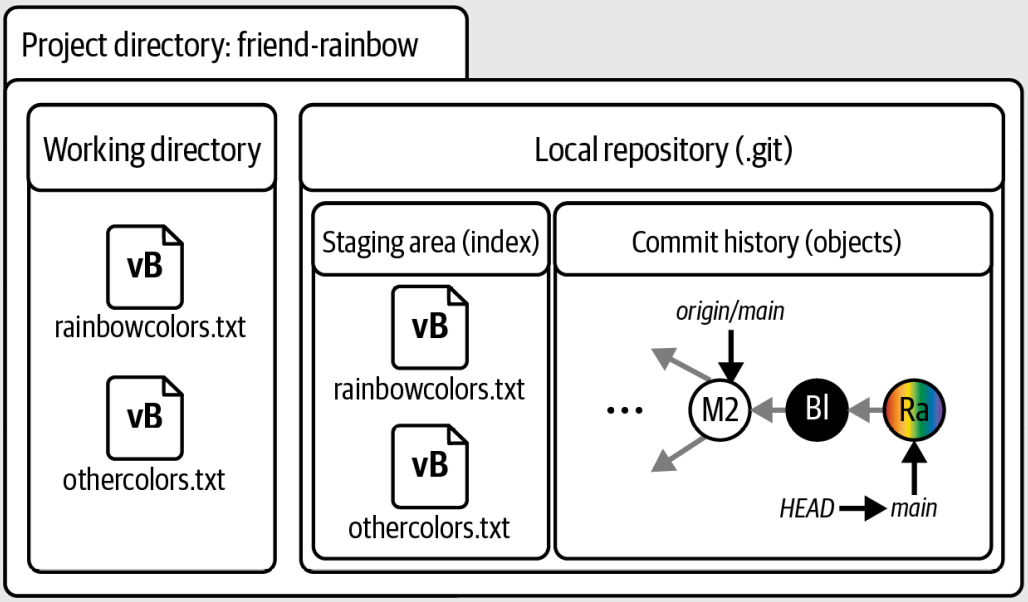
3. friend-rainbow $ **git log**
   commit 51dc6ecb327578cca503abba4a56e8c18f3835e1 (**HEAD -> main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sun Feb 20 10:10:11 2022 +0100
   rainbow
   commit 29bdadd50ddea41c75b476e776b6204a555b3d54
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sun Feb 20 10:07:38 2022 +0100
   black
   commit f10f9725e3319af840a3d891ca8950436a219eb0 (**origin/main, origin/HEAD**)
   Merge: 6ad5c15 9b0a614
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sun Feb 20 09:11:06 2022 +0100
   merge commit 2

What to notice:

- Your friend made the rainbow commit.

This is illustrated in Visualize it 11-9.

[VISUALIZE IT 11-9]



The friend-rainbow project directory after your friend adds the rainbowcolors.txt file to the staging area and makes a commit

What to notice:

- The version of the rainbowcolors.txt file in the staging area is now vB, which is the version that is part of the rainbow commit.

You just saw how your friend was able to add files to and remove them from the staging area in order to craft exactly the commits they wanted. The local main branch in the rainbow repository and the local main branch in the friend-rainbow repository now have divergent development histories. Before your friend continues with the rebasing example, they need to make sure to fetch all the work you have pushed to the remote main branch from the remote repository.

Preparing to Rebase

As you saw in Example Book Project 11-1 earlier in this chapter, to rebase a branch you first have to fetch all the work that has been done on the branch that you want to rebase onto. So, in preparation for rebasing their branch, in Follow Along 11-7 your friend is going to fetch the latest updates from the remote repository.

[FOLLOW ALONG 11-7]

1. friend-rainbow $ **git fetch**

```
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 307 bytes | 153.00 KiB/s, done.
From github.com:gitlearningjourney/rainbow-remote
f10f972..6f2cf36 main -> origin/main
```

2. friend-rainbow $ **git log --all**

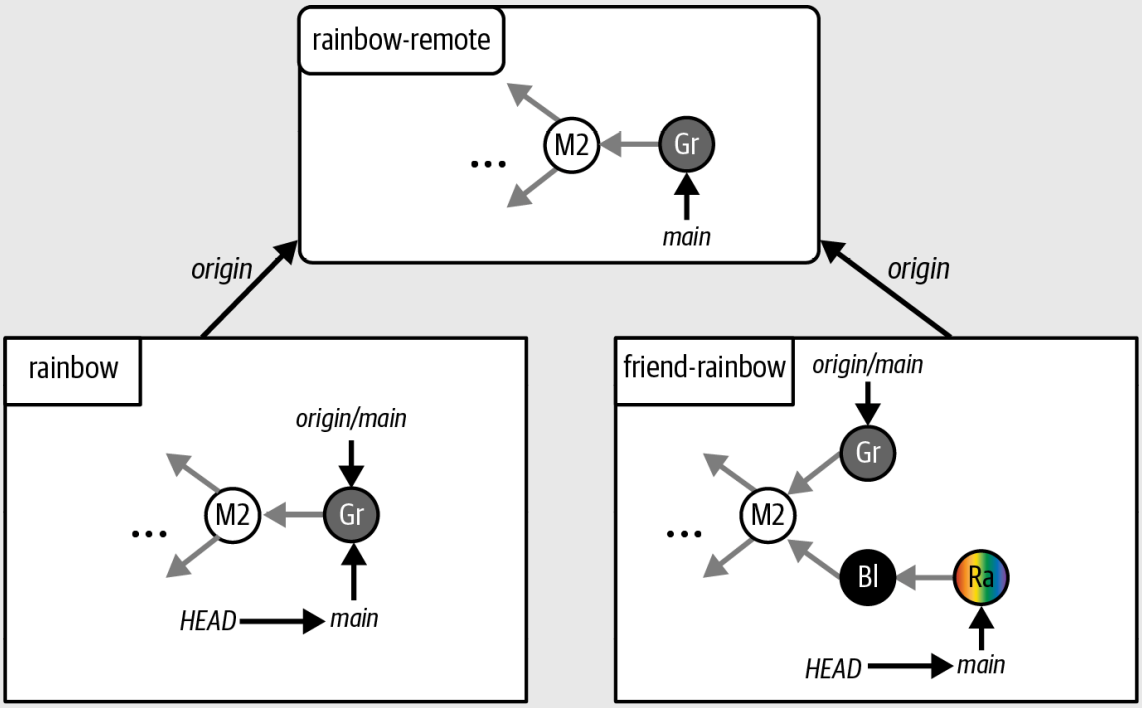
```
commit 51dc6ecb327578cca503abba4a56e8c18f3835e1 (HEAD -> main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:10:11 2022 +0100
rainbow
commit 29bdadd50ddea41c75b476e776b6204a555b3d54
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:07:38 2022 +0100
black
commit 6f2cf3698e6bf9078e8e0340ec9948f590405091 (origin/main, origin/HEAD)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 09:27:58 2022 +0100
gray
```

What to notice:

- Your friend fetched the gray commit from the remote repository, and the origin/main remote-tracking branch was updated to point to it.

This is illustrated in Visualize it 11-10.

[VISUALIZE IT 11-10]



The Rainbow project after your friend updates the friend-rainbow repository by fetching your work from the remote repository

The origin/main remote-tracking branch in the friend-rainbow repository represents the latest version of the remote main branch. Your friend is now ready to rebase their local main branch onto the origin/main remote-tracking branch. Next, we'll go over the stages of the rebase process itself.

The Five Stages of the Rebase Process

This section will walk you through the five stages of the rebase process. To provide a visual illustration of the stages, I'll include Visualize It diagrams previewing the hands-on example that you will carry out later in this chapter, when your friend rebases their local main branch in the friend-rainbow repository onto the origin/main remote-tracking branch.

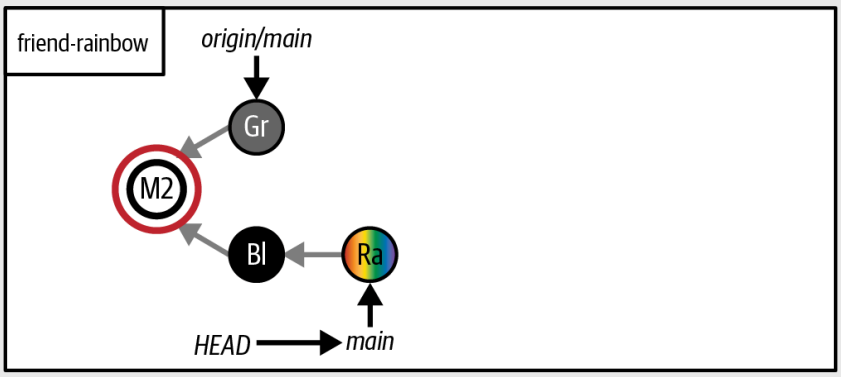
To initiate the rebase process, you use the git rebase command. Git will then carry out the five stages of the process itself; the only time you have to actively get involved is if there are any merge conflicts. We will discuss this situation in further detail in "Rebasing and Merge Conflicts" on page 217.

STAGE 1: FIND THE COMMON ANCESTOR

In the first stage of the process, Git will identify the common ancestor of the two branches involved in the rebase: the branch you're on and the branch you're rebasing onto.

In the Rainbow project example, the branch your friend will be on is the local main branch in the friend-rainbow repository and the branch they will be rebasing onto will be the origin/main remote-tracking branch. The common ancestor will be the M2 merge commit, as shown in Visualize it 11-11.

[VISUALIZE IT 11-11]



Stage 1: Locate the common ancestor of the branches involved in the rebase (here, the M2 merge commit)

STAGE 2: STORE INFORMATION ABOUT THE BRANCHES INVOLVED IN THE REBASE

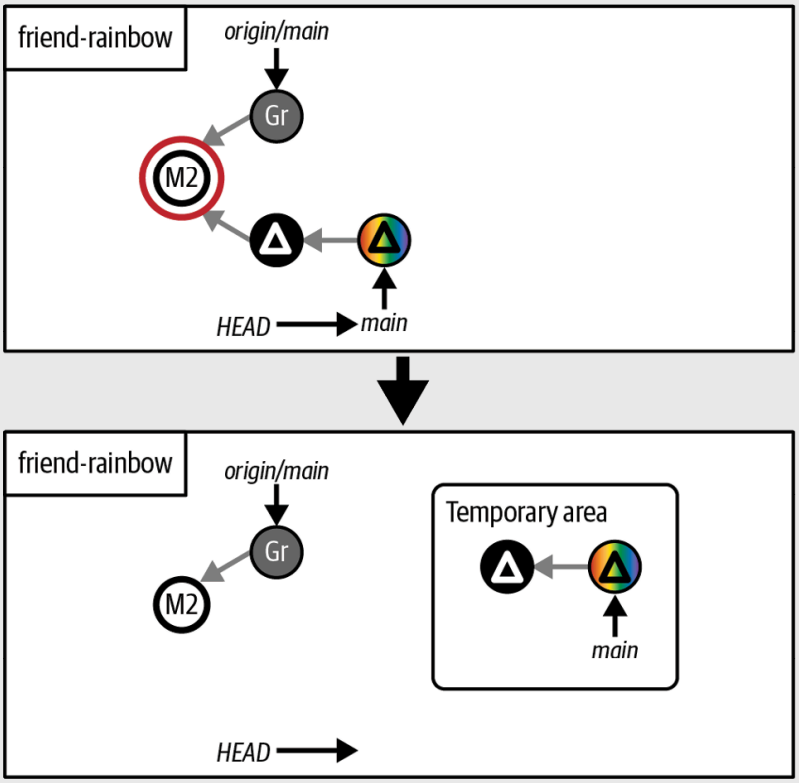
In stage 2, Git will save the changes introduced by each commit of the branch you're on to a temporary area. It will also save additional information in this temporary area, such as which branch you're rebasing onto and where it was pointing when you initiated the rebase.

In the Rainbow project example, the changes introduced by the black commit and the rainbow commit will be saved in the temporary area along with information about the remote main branch, as illustrated in Visualize it 11-12.

[NOTE]

In the following Visualize It diagrams, triangles (△) are used to represent the changes introduced by commits.

[VISUALIZE IT 11-12]



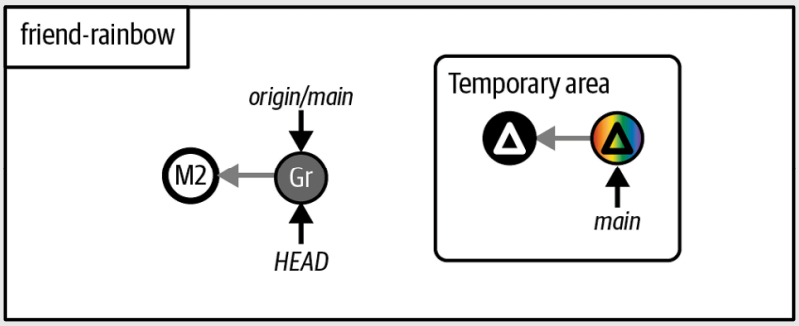
Stage 2: Save information about the rebase in a temporary area

STAGE 3: RESET HEAD

In stage 3, Git will reset HEAD to point to the same commit as the branch you are rebasing onto.

In the Rainbow project example, it will reset HEAD to the same commit that the origin/main remote-tracking branch is pointing to, which is the gray commit, as illustrated in Visualize it 11-13.

[VISUALIZE IT 11-13]



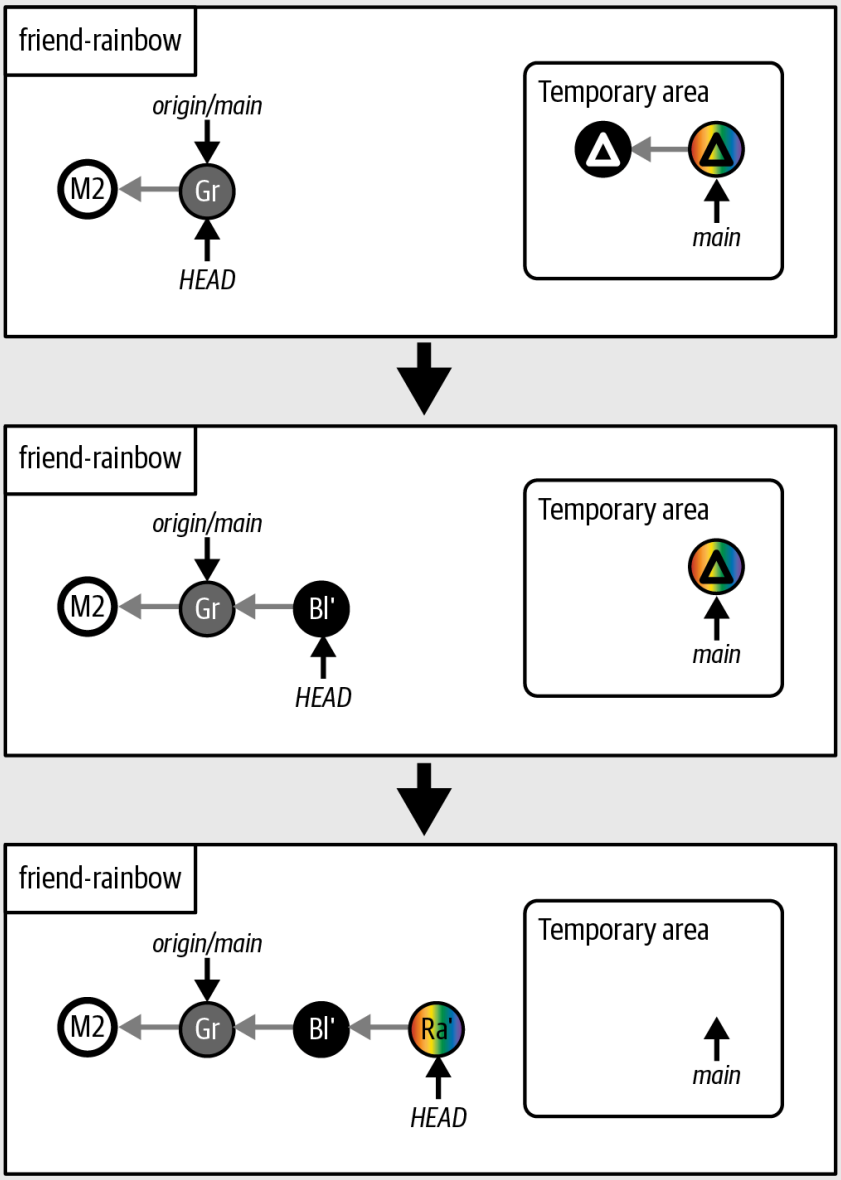
Stage 3: Reset HEAD to the same commit as the branch you are rebasing onto (here, the gray commit)

STAGE 4: APPLY AND COMMIT THE CHANGES

In stage 4, Git will apply the set of changes from each commit in turn, making a commit after it applies each set.

In the Rainbow project example, first it will apply the changes introduced by the black commit and create a new commit, and then it will apply the changes introduced by the rainbow commit and make a new commit, as shown in Visualize it 11-14.

[VISUALIZE IT 11-14]



Stage 4: Apply and commit the changes from each commit (here, the black and rainbow commits)

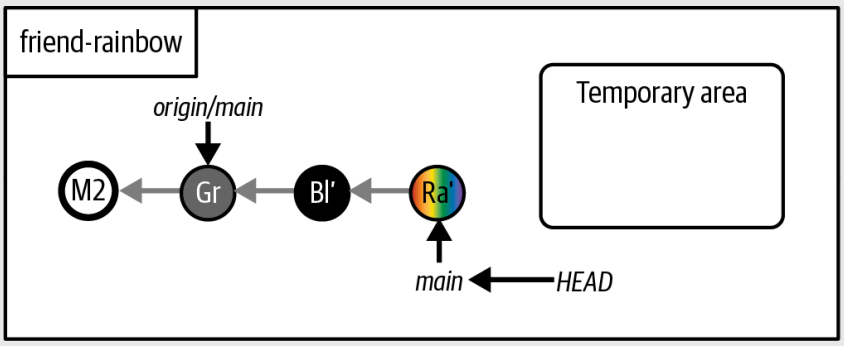
In Visualize it 11-14, the new black commit and the new rainbow commit are represented as Bl' and Ra' (with an apostrophe), because they are entirely new commits.

STAGE 5: SWITCH ONTO THE REBASED BRANCH

In stage 5, Git will make the branch you rebased point to the last commit it reapplies, and it will check out that branch so that HEAD points to it.

In the Rainbow project example, the main branch will point to the new rainbow commit, as illustrated in Visualize it 11-15.

[VISUALIZE IT 11-15]



Stage 5: Switch onto the branch you rebased

This concludes our walkthrough of the five stages of the rebase process. As mentioned at the beginning of this section, Git carries out the entire process itself; all you need to do is initiate it with the git rebase command. I also mentioned that the only time you'll need to get involved in the rebase process is if Git encounters merge conflicts. We'll look briefly at that scenario in the next section.

Rebasing and Merge Conflicts

In Chapter 10 you learned about merge conflicts, which arise when you integrate two branches where different changes have been made to the same parts in the same file(s), or if in one branch a file was deleted that was edited in the other branch.

Git will carry out the entire rebase process independently, unless it encounters merge conflicts. In this case, you must step in and resolve them. The process for resolving merge conflicts while rebasing is similar to the process when doing a three-way merge, with a few small differences.

When resolving merge conflicts in a three-way merge, all the merge conflicts are presented to you at the same time; once you've resolved all the conflicts and added all the updated files to the staging area, you make the final merge commit. By contrast, in the process of rebasing, as Git applies the changes from each commit one by one, it will

pause the process if it encounters merge conflicts in any reapplied commit. This means that you may have to resolve merge conflicts several times when rebasing, depending on how many commits contain merge conflicts.

Once you're done resolving the merge conflicts in a specific commit, you need to add the updated files to the staging area and then instruct Git to resume the rebase process by entering the git rebase command with the --continue option. Git will then continue rebasing the rest of the commits. You don't have to explicitly make any commits, as you do in a three-way merge with conflicts.

As with a merge, if at any point during the process of resolving merge conflicts in a rebase you decide you don't want to continue the rebase process, you can choose to stop or abort the process by using the git rebase command with the --abort option. This will return all your files to the state they were in before the rebase.

[SAVE THE COMMAND]

git rebase --continue

Continue with the rebase process after having resolved merge conflicts

git rebase --abort

Stop the rebase process and go back to the state before the rebase

Now that we've covered what to expect when rebasing, it's time to practice with the Rainbow project.

Rebasing a Branch in Practice

Go to Follow Along 11-8, where your friend will rebase the main branch in their local repository onto the origin/main remote-tracking branch.

[FOLLOW ALONG 11-8]

1. In the friend-rainbow project directory, make a note of the commit hashes for the black commit and the rainbow commit. You may use the git log command for this, which will include the commit hashes in its output. In the example in this book, the commit hashes are:
   Black commit: 29bdadd50ddea41c75b476e776b6204a555b3d54
   Rainbow commit: 51dc6ecb327578cca503abba4a56e8c18f3835e1

2. friend-rainbow $ **git rebase origin/main**
   Auto-merging othercolors.txt
   CONFLICT (content): Merge conflict in othercolors.txt
   error: could not apply 29bdadd... black
   hint: Resolve all conflicts manually, mark them as resolved with
   hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
   hint: You can instead skip this commit: run "git rebase --skip".
   hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
   Could not apply 29bdadd... black

3. friend-rainbow $ **git status**
   interactive rebase in progress; onto 6f2cf36
   Last command done (1 command done):
   pick 29bdadd black
   Next command to do (1 remaining command):
   pick 51dc6ec rainbow
   (use "git rebase --edit-todo" to view and edit)
   You are currently rebasing branch 'main' on 'bcb1dc0'.
   (fix conflicts and then run "git rebase --continue")
   (use "git rebase --skip" to skip this patch)
   (use "git rebase --abort" to check out the original branch)
   Unmerged paths:
   (use "git restore --staged <file>..." to unstage)
   (use "git add <file>..." to mark resolution)
   both modified: othercolors.txt
   no changes added to commit (use "git add" and/or "git commit -a")

What to notice:

- The rebase operation was interrupted because when Git was applying the changes that were included in the black commit, it encountered a merge conflict.
- In step 2, the git rebase command hints to you that you must Resolve all conflicts manually, mark them as resolved with "git add/rm <conflicted_files>", then run "git rebase --continue".
- In step 3, the git status command also shows you information about the rebase and which files you need to resolve merge conflicts in.

In Follow Along 11-9, your friend is going to apply the steps you learned in Chapter 10 to resolve the merge conflicts. Once they're done, they will execute the git rebase --continue command.

[FOLLOW ALONG 11-9]

1. Carry out step 1 of resolving merge conflicts, which is to choose what to keep, edit the content, and remove the conflict markers. You will keep all the changes from both branches. Keep the "Gray is not a color in the rainbow." sentence above the "Black is not a color in the rainbow." sentence. Make sure to save the othercolors.txt file when you're done making your edits.

2. friend-rainbow $ **git add othercolors.txt**

3. friend-rainbow $ **git status**
   interactive rebase in progress; onto 6f2cf36
   Last command done (1 command done):
   pick 29bdadd black
   Next command to do (1 remaining command):
   pick 51dc6ec rainbow
   (use "git rebase --edit-todo" to view and edit)
   You are currently rebasing branch 'main' on 'bcb1dc0'.
   (all conflicts fixed: run "git rebase --continue")
   Changes to be committed:
   (use "git restore --staged <file>..." to unstage)
   modified: othercolors.txt

4. friend-rainbow $ **git rebase --continue**

5. Accept the default commit message in the Vim editor, and exit the editor. As you learned in Chapter 9, to do this you must press the Escape key, type **:wq**, and press Enter. You will see the following output:
   [detached HEAD e055f2b] black
   1 file changed, 3 insertions(+), 1 deletion(-)
   Successfully rebased and updated refs/heads/main.

6. friend-rainbow $ **git log**
   commit 7c09136bcbfdd9f638ed13c6653e06451579d21c (**HEAD -> main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sun Feb 20 10:10:11 2022 +0100
   rainbow
   commit e055f2bc66aed1f3627041900a8c825c7a875206

[FOLLOW ALONG 11-9]

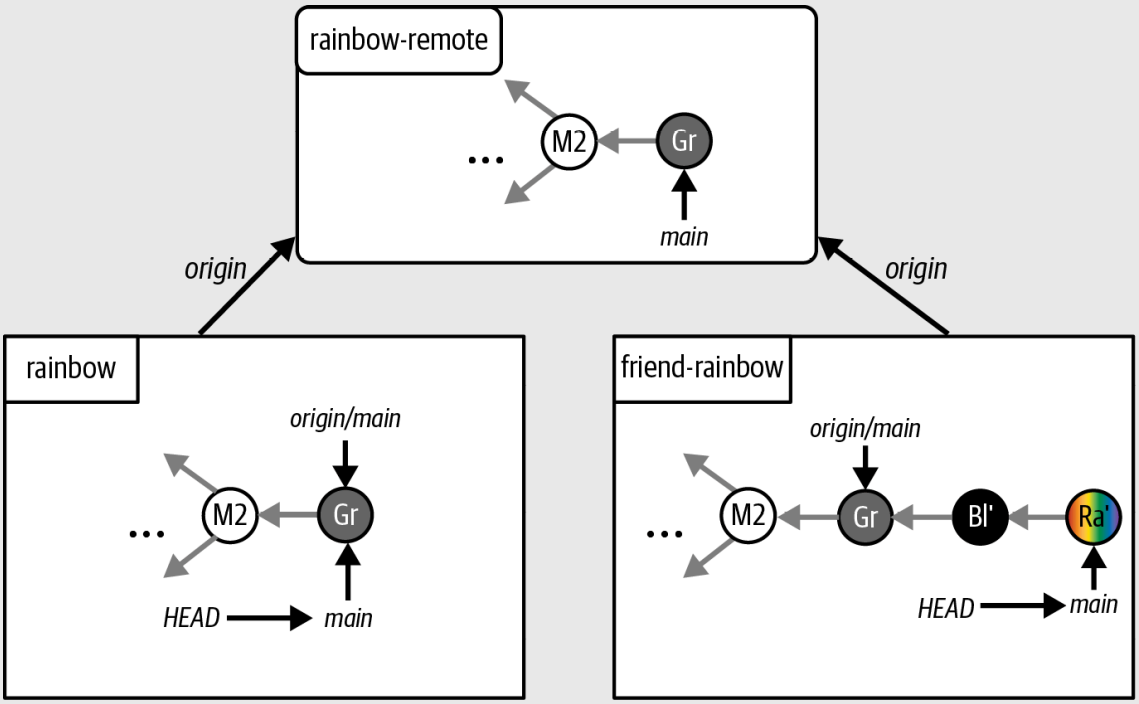
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:07:38 2022 +0100
black
commit 6f2cf3698e6bf9078e8e0340ec9948f590405091 (**origin/main, origin/HEAD**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 09:27:58 2022 +0100
gray

What to notice:

- In step 3, the git status output informs you all conflicts fixed: run "git rebase --continue".
- In step 6, the git log output indicates that the rebase process created a new rainbow commit and a new black commit. The commit hashes are new, as are the timestamps (the date and time of each commit).

The result of this process is illustrated in Visualize it 11-16.

[VISUALIZE IT 11-16]



The Rainbow project after your friend rebases their local main branch onto the origin/main remote-tracking branch

What to notice:

- We represent the new black commit and the new rainbow commit as Bl′ and Ra′ (with an apostrophe).
- In the friend-rainbow repository, the gray commit, the new black commit, and the new rainbow commit form a linear project history. In other words, there is no merge commit.

Figure 11-7 compares the commit hashes of the old black and rainbow commits and the new black and rainbow commits in this book. The commit hashes in your repositories will be different from the ones in this book because commit hashes are unique.



FIGURE 11-7

The commit hashes for the old black and rainbow commits are different from the commit hashes for the new black and rainbow commits

From this exercise, you have observed that rebasing rewrites history—and that brings us to the golden rule of rebasing.

The Golden Rule of Rebasing

As you saw in the Rainbow project example, rebasing creates entirely new commits. This means that rebasing changes the commit history. You must always be careful when you change the commit history because it can lead to complications in your project, especially when you're working with other people.

The golden rule of rebasing states that *you should not rebase a branch that other people may have based work on*. For example, if you have pushed a branch to the remote repository, then it is considered a public branch. This means that other collaborators may also be working on this branch in their local repositories, or they may be pushing work to this branch in the remote repository.

In this case, you should refrain from rebasing the branch. To explore the importance of the golden rule of rebasing, let's take a look at Example Book Project 11-3, in which we revisit the situation discussed in Example Book Project 11-1.

EXAMPLE BOOK PROJECT 11-3

In Example Book Project 11-1, I introduced a situation where there were two commits on the main branch (commits A and B) when I made the chapter_five branch and my coauthor made the chapter_six branch. Subsequently, my coauthor merged commit C from their chapter_six branch into their main branch and pushed the updated main branch to the remote expository. At the same time, I added commit D to the chapter_five branch in my local repository. This initial scenario is illustrated in Figure 11-8.

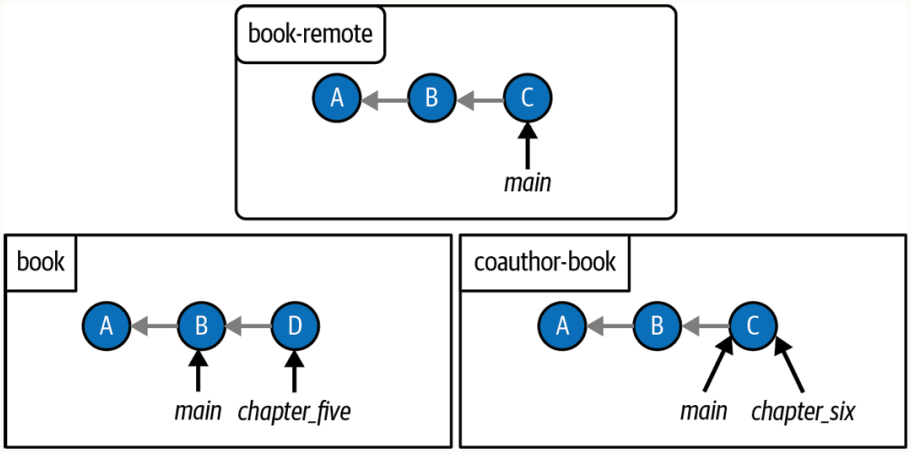


FIGURE 11-8

The Book project when the local chapter_five branch and the remote main branch have diverged

Now, suppose that I push the chapter_five branch to the remote repository, and then, in violation of the golden rule of rebasing, I also pull commit C from the remote repository and update my local main branch because I plan to rebase my chapter_five branch onto the latest version of the main branch. This situation is depicted in Figure 11-9.

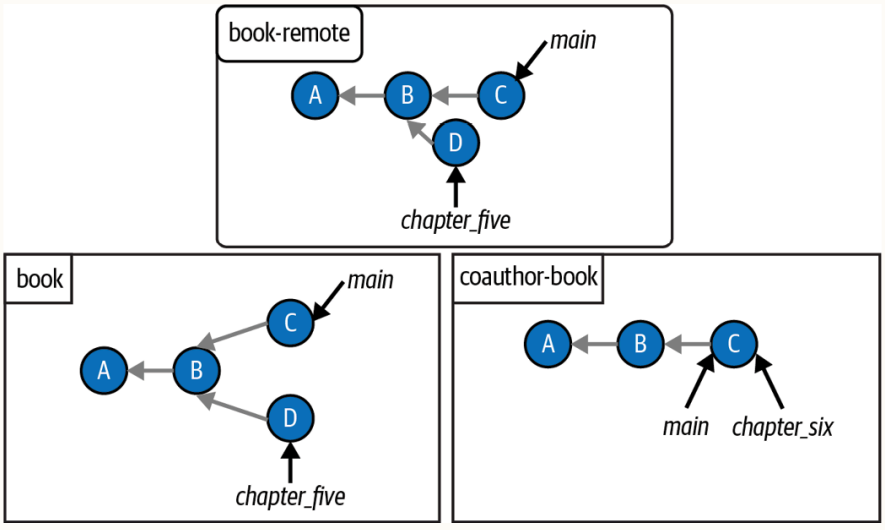


FIGURE 11-9

The Book project after I push the chapter_five branch to the remote repository and pull the commits on the main branch

I don't realize my mistake, and I go ahead with my plan to rebase the chapter_five branch onto the main branch. Figure 11-10 shows the state of the remote and local repositories after the rebase. D′ in the book repository represents the re-created commit D after the rebase.

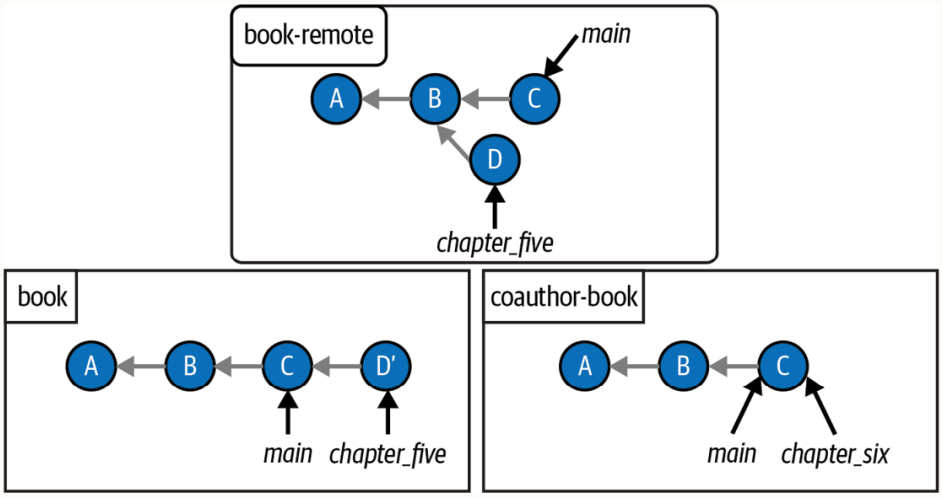


FIGURE 11-10

The Book project after I rebase the chapter_five branch onto the main branch

I have not yet pushed my updated chapter_five branch to the remote repository. Now let's assume my coauthor sees the remote chapter_five branch in the remote repository and decides to contribute to it. They pull the chapter_five branch into their coauthor-book repository, add commits E and F to it, and then push them up to the remote repository. This situation is illustrated in Figure 11-11.

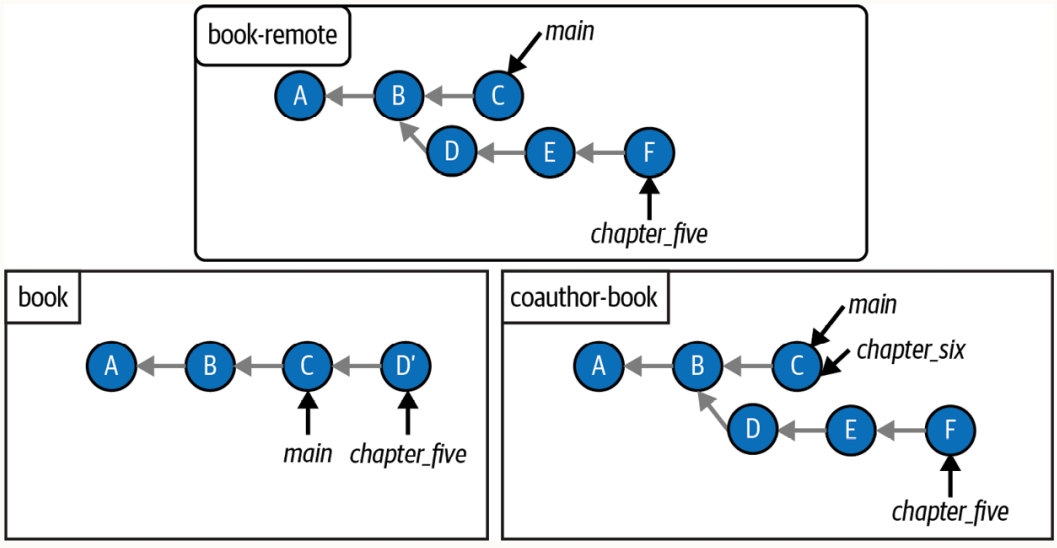


FIGURE 11-11

The Book project after my coauthor contributes to the remote chapter_five branch

When I try to push my chapter_five branch to the remote repository I will receive the following error:

error: failed to push some refs to 'github.com:gitlearningjourney/rainbow-remote.git'

hint: Updates were rejected because the tip of your current branch is behind its remote counterpart. Integrate the remote changes (e.g. 'git pull ...') before pushing again. See the 'Note about fast-forwards' in 'git push --help' for details.

This error informs me that I am not able to push my changes to the remote repository. As you can see in Figure 11-11, the commit histories of the remote chapter_five branch and the local chapter_five branch no longer contain the same commits: the remote chapter_five branch is made up of commits A, B, D, E, and F, while the local chapter_five branch is made up of commits A, B, C, and D'.

This is a sticky situation, because now my coauthor and I have no easy way of resolving this issue. We will have to communicate to troubleshoot and see how we can resolve the disparities between our branches.

Example Book Project 11-3 illustrates why you should follow the golden rule of rebasing when using the rebase operation. To conclude, you may safely rebase a branch if:

- You have a local branch that has never been pushed to the remote repository.
- You have a local branch that you've pushed to a remote repository that you're 100% sure nobody has based work on or contributed to.

That's it. If there is a possibility that someone else has worked on the branch, then it is recommended that you avoid rebasing.

Syncing the Repositories

To make sure the repositories in the Rainbow project are in sync, your friend will have to push their changes to the remote repository and you will have to pull the changes down into the rainbow repository. Go to Follow Along 11-10 to do this now.

[FOLLOW ALONG 11-10]

1. friend-rainbow $ **git push**

```
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 652 bytes | 652.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:gitlearningjourney/rainbow-remote.git
6f2cf36..7c09136 main -> main
```

2. friend-rainbow $ **git log**

```
commit 7c09136bcbfdd9f638ed13c6653e06451579d21c (HEAD -> main,
origin/main, origin/HEAD)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:10:11 2022 +0100
rainbow
commit e055f2bc66aed1f3627041900a8c825c7a875206
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:07:38 2022 +0100
black
```

3. Go to the command line window where you're in the rainbow repository to execute the command in step 4.

4. rainbow $ **git pull**

```
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 6 (delta 1), reused 6 (delta 1), pack-reused 0
Unpacking objects: 100% (6/6), 632 bytes | 105.00 KiB/s, done.
From github.com:gitlearningjourney/rainbow-remote
6f2cf36..7c09136 main -> origin/main
Updating 6f2cf36..7c09136
Fast-forward
othercolors.txt | 4 +++-
rainbowcolors.txt | 2 +-
```

[FOLLOW ALONG 11-10]

2 files changed, 4 insertions(+), 2 deletions(-)

5 rainbow $ **git log**

commit 7c09136bcbfdd9f638ed13c6653e06451579d21c (**HEAD -> main, origin/main**)

Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sun Feb 20 10:10:11 2022 +0100

rainbow

commit e055f2bc66aed1f3627041900a8c825c7a875206

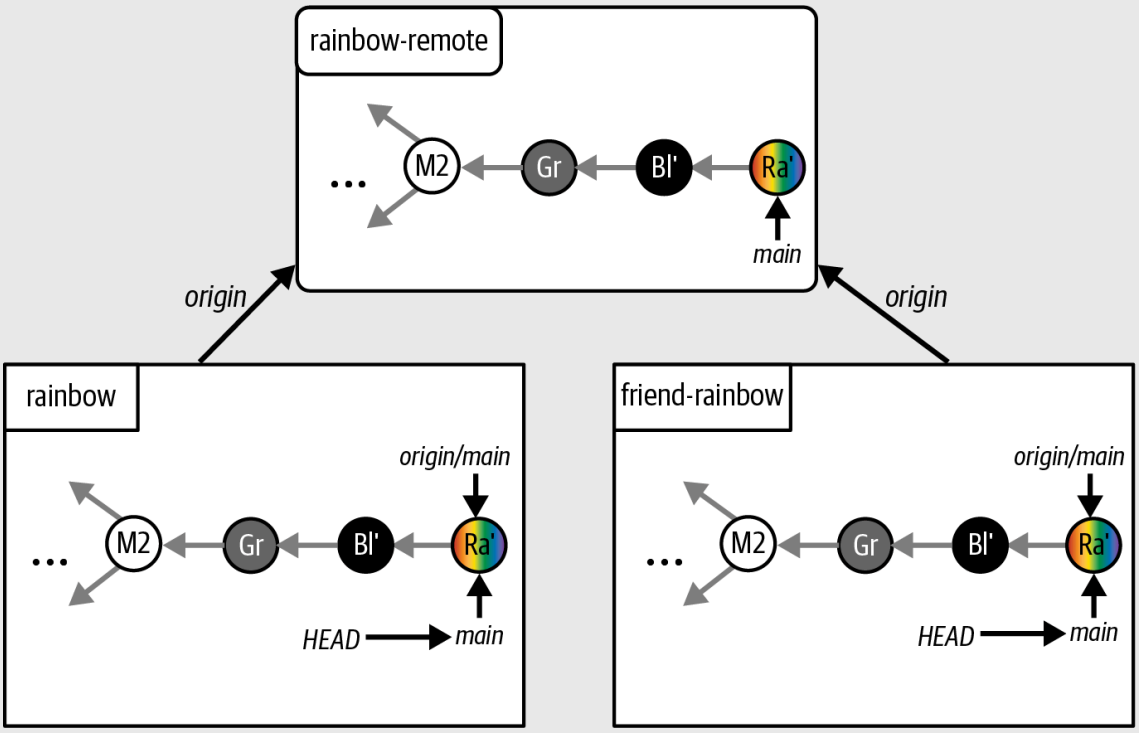
Author: annaskoulikari <gitlearningjourney@gmail.com>

Date: Sun Feb 20 10:07:38 2022 +0100

black

All three repositories in the Rainbow project are now in sync, as illustrated in Visualize it 11-17.

[VISUALIZE IT 11-17]

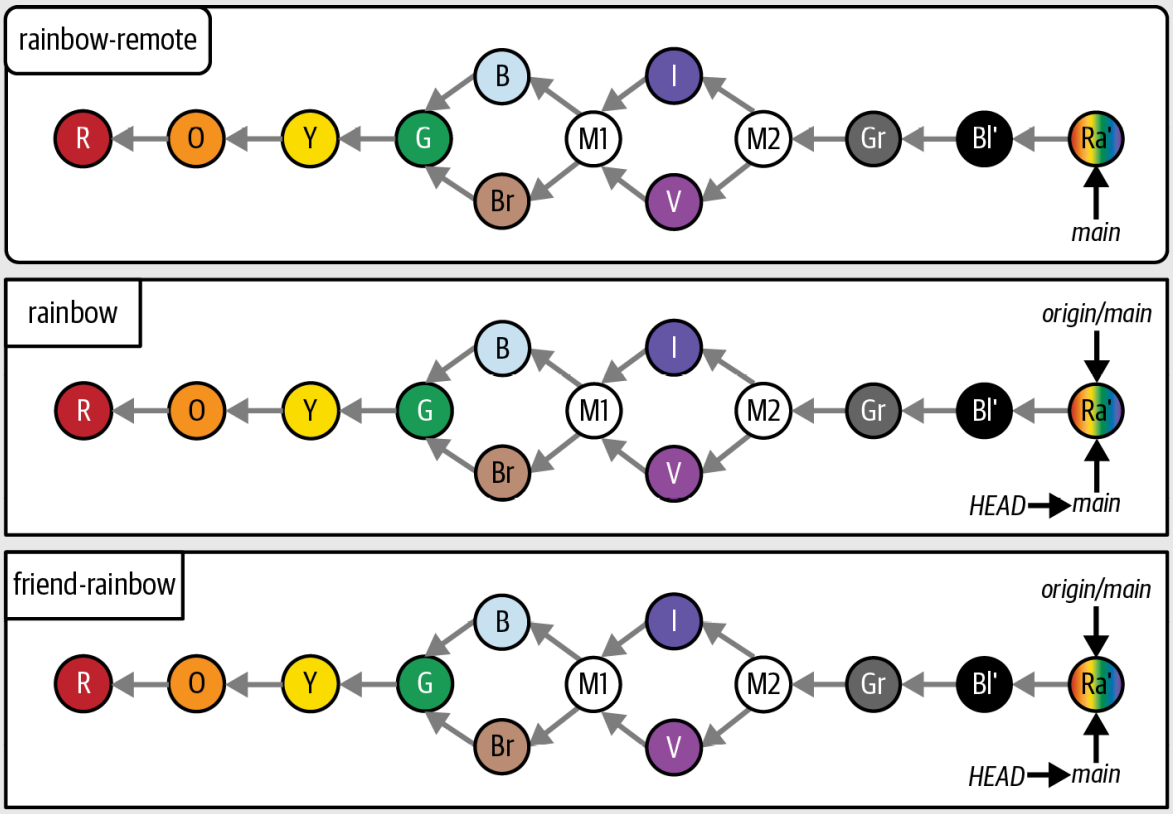


The Rainbow project after your friend pushes their local main branch to the remote repository and you pull from the remote repository to update your local main branch

State of the Local and Remote Repositories

Visualize it 11-18 shows the state of the local and remote repositories in the Rainbow project with all the commits that were made from Chapter 1 all the way to the end of this chapter.

[VISUALIZE IT 11-18]



The Rainbow project at the end of Chapter 11 with all the commits since Chapter 1

Summary

In this chapter you learned about rebasing—the second way of integrating changes from one branch to another in Git—and how it can be used to avoid three-way merges and merge commits. You learned about the five stages of the rebase process that Git carries out, then you practiced rebasing with an example in the Rainbow project. During the process, you also practiced resolving merge conflicts.

Because rebasing rewrites the commit history, I introduced the golden rule of rebasing, which states that you should not rebase branches that other collaborators may have based work on. Additionally, you learned how to unstage files, or remove modified files from the staging area, so you can customize exactly what you include in a commit.

To this point, we have covered how you can integrate changes from one branch into another when working with a local repository through either merging or rebasing. Next, in

Chapter 12, we're going to explore a helpful collaboration tool for Git projects that allows you to integrate changes remotely, called a pull request.

Previous chapter
< Chapter 10. Merge Conflicts

Next chapter
Chapter 12. Pull Requests (Merge Requests) >